

Introduction to Computer Systems Design

CS3031 · IIT Madras · Week 4

Module 2: Combinational Logic Elements

Detailed Study Notes

Combinational Building Blocks

- ▶ Why build with blocks: MSI, abstraction, “Lego blocks”
- ▶ XOR / XNOR and the equality comparator (1-bit and N -bit)
- ▶ Magnitude comparison ($A > B$, $A < B$) from the annotated slides
- ▶ Decoders: definition, gate design, address and opcode decoding, implementing any function
- ▶ Encoders, and why they fail on multiple inputs
- ▶ Priority encoders and don't-care conditions
- ▶ Multiplexers: 2:1, 4:1, $\log_2 N$ select lines, mux as a function generator
- ▶ Beyond: half/full adders and PLAs (listed on the slides)
- ▶ 32 numericals, 20 subjective Qs, 40 MCQs

Based on lecture slides by Ayon Chakraborty, Department of CSE, IIT Madras. Boxes marked *Beyond the Slides* add standard textbook context that is not on the slides. Handwritten annotations on the slides have been interpreted and typeset; where they were ambiguous, this is flagged.

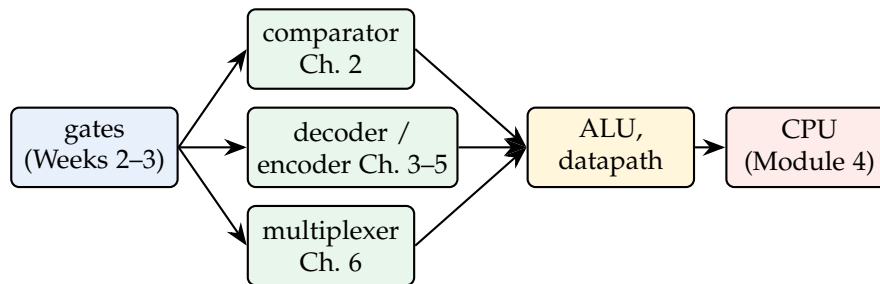
Contents

How to Use These Notes	3
I Module 2 — Combinational Logic Elements	4
1 Combinational Building Blocks	5
1.1 Where we are	5
1.2 Why group gates into blocks?	5
1.3 Functionalities you would expect	6
1.4 Standard blocks	6
2 Comparators	7
2.1 XOR and XNOR	7
2.1.1 XOR and the half adder (handwritten notes)	7
2.2 1-bit equality comparator	8
2.3 Comparing two N -bit numbers	8
2.3.1 Equality: all bits must match	8
2.4 Magnitude comparison: $A > B$ and $A < B$	9
2.4.1 1-bit	9
2.4.2 2-bit, derived	9
3 Decoders	11
3.1 Definition	11
3.2 What decoders are used for	12
3.2.1 Address decoding	12
3.2.2 Instruction (opcode) decoding	12
3.3 A decoder can implement any function	13
4 Encoders and Priority Encoders	15
4.1 Encoder	15
4.1.1 Deriving the equations	15
4.1.2 The hidden assumption: exactly one input is active	16
4.2 Priority encoder	16
4.2.1 Don't-care conditions in the <i>inputs</i>	16
4.2.2 Deriving the equations	16
5 Multiplexers	18

5.1	Definition	18
5.1.1	How it works	18
5.2	Bigger multiplexers	19
5.2.1	Mux trees	19
5.3	A multiplexer can implement any function	20
6	Beyond the Slides: Full Adders and PLAs	21
6.1	Half adder (recap from Chapter 2)	21
6.2	Full adder	21
6.2.1	Ripple-carry adder	21
6.3	Programmable Logic Arrays (PLAs)	22
II	Practice	23
7	Numerical Problems with Worked Solutions	24
7.1	XOR and comparators	24
7.2	Decoders	25
7.3	Encoders	27
7.4	Multiplexers	27
7.5	Adders and PLAs (beyond)	28
8	Subjective Questions with Answers	30
9	Multiple-Choice Questions	34
9.1	Questions	34
9.2	Answers and Explanations	39
A	Formula Sheet, Glossary and Revision	42
A.1	Formula sheet	42
A.2	Glossary	42
A.3	Revision checklist	42

How to Use These Notes

Weeks 2–3 gave you the algebra and the minimisation tools. Week 4 starts **Module 2**: using those tools once, carefully, to design a small library of standard blocks. From then on, larger designs (the ALU, the CPU) are drawn with these blocks rather than with individual gates — the same abstraction step as “transistors → gates” in Week 1, one level up.



The coloured boxes

Intuition

The mental picture behind an idea.

Key Idea

The one sentence to remember.

Formula

A result with its derivation; sizes/counts in grey brackets like [rows] or [gates].

Common Pitfall

A common exam mistake.

Beyond the Slides

Standard background not on the slides (Mano; Harris & Harris).

Notation

$A \oplus B$	XOR: $A\bar{B} + \bar{A}B$ (1 when inputs differ)
$A \odot B, \bar{A} \oplus \bar{B}$	XNOR: $AB + \bar{A}\bar{B}$ (1 when inputs are equal)
$A_{N-1} \dots A_1 A_0$	N -bit number, A_{N-1} = MSB (see the note on slide indexing in Ch. 2)
$n:2^n$ decoder	n inputs, 2^n outputs
$2^n:n$ encoder	2^n inputs, n outputs
$N:1$ mux	N data inputs, $\log_2 N$ select inputs, 1 output
X	don't care

Part I

Module 2 — Combinational Logic Elements

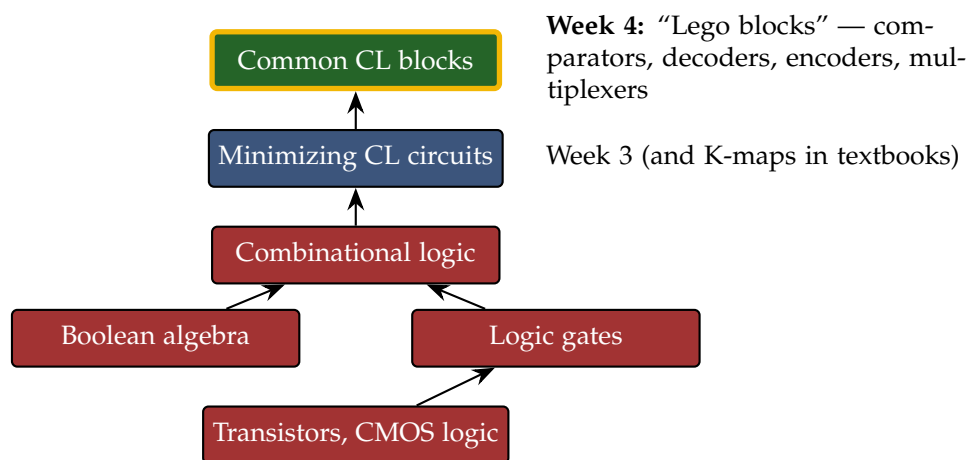
Chapter 1

Combinational Building Blocks

1.1 Where we are

From the slides:

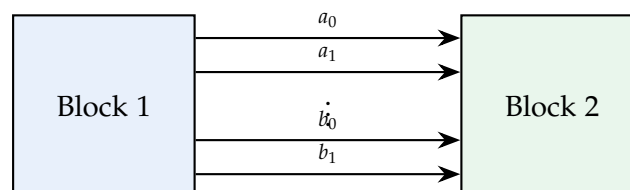
Recall Again The transformation hierarchy, zoomed into Logic: Transistors, CMOS logic → Boolean Algebra and Logic Gates → Combinational Logic (CL) → Minimizing CL Circuits → **Common CL Blocks** (handwritten: “Lego blocks”; “formal ways to minimize (K-maps)”).



1.2 Why group gates into blocks?

From the slides:

Combinational Building Blocks Combinational logic is often grouped into larger building blocks to build more complex systems. Typically, tens to hundreds of transistors — *used to be* called Medium Scale Integration (MSI). Common building blocks for digital systems hide the unnecessary gate-level details to emphasize the function of the building block. *Functionalities you would expect?* (Handwritten: Block 1 with outputs A, B feeding Block 2 through wires $a_0 \dots, b_0 \dots$ — blocks talk to blocks.)



Each block is described by its *function* (what the outputs mean), not by its gates. That is the MSI idea: a chip or a library cell that “compares” or “selects”.

Intuition : Lego

A Lego brick has a standard interface (the studs) and hides its insides. You build a castle from bricks without thinking about plastic moulding. Combinational blocks are the bricks of digital design: once a comparator is designed and verified, you just use it. Historically (1970s), each such block was a separate MSI chip in the TTL 7400 family; today they are cells in a design library.

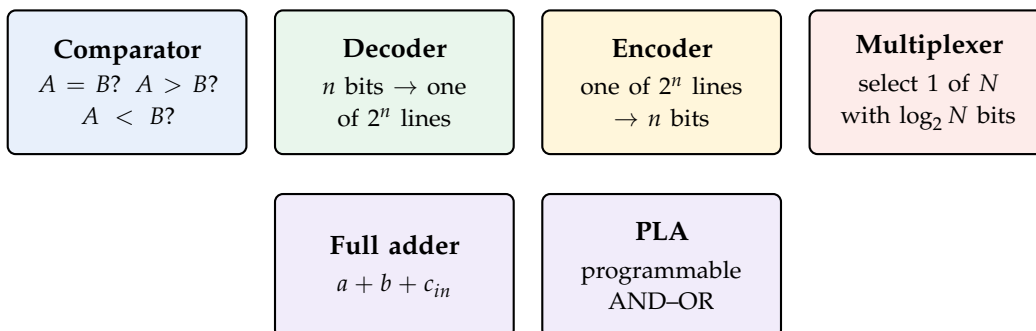
1.3 Functionalities you would expect

The slide leaves this as a question. Think about what a processor has to do to data:

Need	Example in a computer	Block
Check equality / order	“branch if equal”, “is $x < y$?”	comparator
Turn a code into “which one”	address $\rightarrow$ one memory row; opcode $\rightarrow$ one operation	decoder
Turn “which one” into a code	which key is pressed $\rightarrow$ key code; highest-priority request	(priority) encoder
Choose among inputs	ALU picks the result of + or AND; register selection	multiplexer
Arithmetic	add, subtract	(full) adder
Arbitrary logic, reconfigurable	glue logic, prototypes	PLA / programmable logic

1.4 Standard blocks**From the slides:**

Combinational Building Blocks (list) Some standard blocks include: Comparators; Decoders / Encoders; Multiplexers; Full adders; PLAs (Programmable Logic Arrays). (Handwritten: comparators “ $x = y$, $x < y$, $x > y$ ” with examples; decoders “ n bits represent 2^n different entities”, “10-bit $\rightarrow$ 1024 memory locations”; multiplexers “limit one of the # of gates (MUX) / selection”; PLAs “re-configurable, programmable hardware logic”.)

**Key Idea**

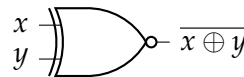
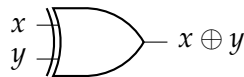
The slides cover comparators, decoders, encoders (including priority encoders) and multiplexers in detail. Full adders and PLAs are listed but not developed; Chapter 6 gives a short introduction because the ALU of Module 2 is built from adders.

Comparators

2.1 XOR and XNOR

From the slides:

Equality Comparator Using XOR Truth table with columns $x, y, x \oplus y, \overline{x \oplus y}$: rows $00 \rightarrow 0, 1$; $01 \rightarrow 1, 0$; $10 \rightarrow 1, 0$; $11 \rightarrow 0, 1$. **XOR**: $x\bar{y} + \bar{x}y$ — similar to OR gate, except when inputs are 1; used for comparisons, error checking, etc. **XNOR**: $xy + \bar{x}\bar{y}$ — complemented version of XOR. One bit equality comparator? How about comparing two N -bit numbers? (Handwritten: XOR column labelled “carry?”/“sum bit XOR”, AND labelled “carry bit AND” — “half adder”, “towards building a circuit for addition”; “comp., error check”.)



x	y	$x \oplus y$	$\overline{x \oplus y}$ (XNOR)	$x + y$ (OR)	$x \cdot y$ (AND)
0	0	0	1	0	0
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	1	1	1

- **XOR** (exclusive OR): 1 when *exactly one* input is 1 — i.e. when the inputs **differ**. It is OR except in the last row (“similar to OR except when both inputs are 1”).
- **XNOR**: 1 when the inputs are **equal**. It is the complement of XOR.

Both expressions are just canonical SOPs: $\text{XOR} = \sum m(1,2) = x\bar{y} + \bar{x}y$, $\text{XNOR} = \sum m(0,3) = \overline{x\bar{y} + \bar{x}y}$.

Intuition : XOR as a “difference detector” and “controlled inverter”

$x \oplus 0 = x$ and $x \oplus 1 = \bar{x}$. So XOR with a control bit c either passes x unchanged ($c = 0$) or inverts it ($c = 1$). This reading explains many uses: error detection (parity), the three-switch bulb of Week 3, and the “subtract” trick in an ALU (invert B when subtracting).

2.1.1 XOR and the half adder (handwritten notes)

Adding two 1-bit numbers $a + b$ gives a 2-bit result (c, s) :

a	b	carry c	sum s	$a + b$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	0	$2 = 10_2$

$$s = a \oplus b, \quad c = a \cdot b$$

The sum column is exactly XOR and the carry column is AND — the handwritten “towards building a circuit for addition”. Chapter 6 extends this to the full adder.

2.2 1-bit equality comparator

Two bits are equal iff their XNOR is 1:

$$E = \overline{x \oplus y} = xy + \overline{x}\overline{y}, \quad D = x \oplus y \text{ ("different")}$$

One XNOR gate (or an XOR followed by an inverter) is a 1-bit equality comparator.

2.3 Comparing two N -bit numbers

From the slides:

Comparing Two N -bit Numbers Inputs: $A_0A_1A_2A_3$, $B_0B_1B_2B_3$. Example: 11_{10} and 9_{10} , i.e. 1011_2 and 1001_2 . A black box with 8 inputs and output $A \neq B$; the next slide opens the box: four "comp" (XOR) blocks, one per bit pair A_iB_i , feeding a tree of OR gates that outputs $A \neq B$. (Handwritten: " $A \rightarrow A_3A_2A_1A_0$, MSB ... LSB"; a 2-bit table of a_1a_0 , b_1b_0 with $A > B$ and $A < B$ columns; expressions like $a_1\overline{b_1} + \dots$)

Indexing note. The slide lists inputs as $A_0A_1A_2A_3$, while the handwriting says " $A \rightarrow A_3A_2A_1A_0$, MSB ... LSB". For *equality*, bit order doesn't matter. For *magnitude* comparison it does; these notes use $A_3 = \text{MSB}$.

2.3.1 Equality: all bits must match

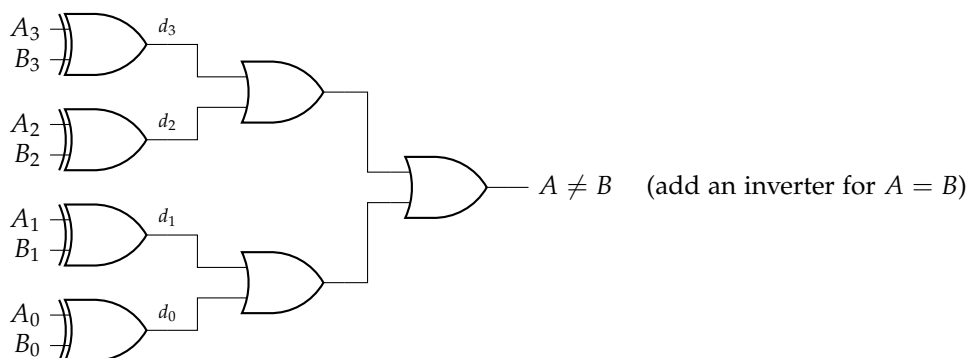
$A = B$ iff $A_i = B_i$ for every i :

Formula : N -bit equality

$$E = \prod_{i=0}^{N-1} \overline{A_i \oplus B_i} = \overline{\sum_{i=0}^{N-1} (A_i \oplus B_i)}, \quad D = (A \neq B) = \sum_{i=0}^{N-1} (A_i \oplus B_i).$$

Cost: N XOR gates + an N -input OR (or a tree of $N - 1$ two-input ORs).

Derivation. "All equal" = AND of the N per-bit XNORs. By De Morgan, $\prod \overline{d_i} = \overline{\sum d_i}$ with $d_i = A_i \oplus B_i$: "no bit differs". The slide builds the complement $D = \sum d_i$: "some bit differs".



Worked Example : 11 vs 9

$A = 1011$, $B = 1001$ (A_3 first).

bit i	3	2	1	0
A_i	1	0	1	1
B_i	1	0	0	1
$d_i = A_i \oplus B_i$	0	0	1	0

$D = 0 + 0 + 1 + 0 = 1$: $A \neq B$ (they differ in bit 1). $E = 0$.

Common Pitfall

A 4-bit comparator's truth table has $2^8 = 256$ rows; for 32-bit numbers, 2^{64} . Nobody writes these tables. The comparator is designed *structurally* — per-bit blocks plus a combining tree — which is exactly why building blocks matter.

2.4 Magnitude comparison: $A > B$ and $A < B$

The handwritten notes extend the comparator to ordering. Compare from the most significant bit down, like comparing words in a dictionary.

2.4.1 1-bit

$$G_1 = (a > b) = a\bar{b}, \quad L_1 = (a < b) = \bar{a}b, \quad E_1 = \overline{a \oplus b}.$$

Exactly one of G_1, L_1, E_1 is 1 for every input.

2.4.2 2-bit, derived

$A = a_1a_0, B = b_1b_0$. $A > B$ iff either the high bits decide ($a_1 > b_1$), or the high bits are equal and the low bits decide:

$$G = a_1\bar{b}_1 + e_1 a_0\bar{b}_0, \quad L = \bar{a}_1b_1 + e_1 \bar{a}_0b_0, \quad E = e_1e_0,$$

with $e_i = \overline{a_i \oplus b_i}$.

$A \setminus B$	00	01	10	11
00	E	L	L	L
01	G	E	L	L
10	G	G	E	L
11	G	G	G	E

Of the 16 input combinations, 4 give E, 6 give G (the entries below the diagonal) and 6 give L. You can check that the formula for G is 1 exactly in the 6 G cells.

Formula : N -bit magnitude comparator (A_{N-1} = MSB)

$$G = A_{N-1}\bar{B}_{N-1} + e_{N-1}A_{N-2}\bar{B}_{N-2} + e_{N-1}e_{N-2}A_{N-3}\bar{B}_{N-3} + \cdots + e_{N-1}\cdots e_1 A_0\bar{B}_0$$

L : the same with $\bar{A}_iB_i$ in place of $A_i\bar{B}_i$. $E = e_{N-1}\cdots e_0$. Also $L = \overline{G + E}$.

Derivation. Let k be the highest bit position where A and B differ. Bits above k are equal ($e_i = 1$ for $i > k$), and since bit k has weight $2^k > \sum_{j < k} 2^j = 2^k - 1$, bit k alone decides the order. So $A > B$ iff at that k , $A_k = 1$ and $B_k = 0$. The k -th term of G encodes exactly this.

Worked Example : $11 > 9$

$A = 1011$, $B = 1001$. $e_3 = 1$, $e_2 = 1$, bit 1: $A_1 \overline{B_1} = 1 \cdot 1 = 1$. So the third term $e_3 e_2 A_1 \overline{B_1} = 1$ and $G = 1$. ✓

Beyond the Slides : comparing with a subtractor

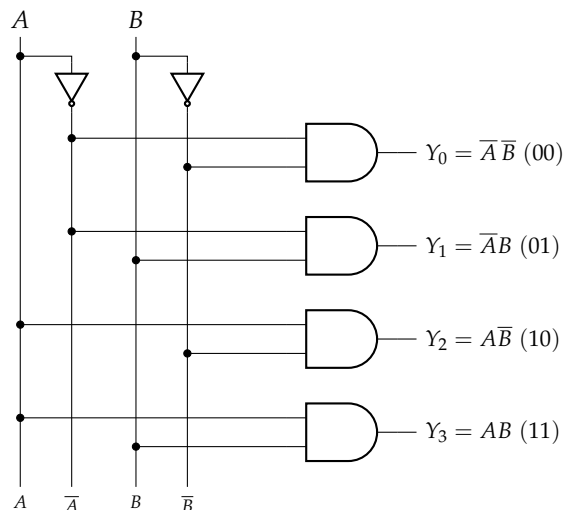
In a processor, $A < B$ is often computed by an ALU subtraction $A - B$ and inspecting the sign (and overflow) of the result, re-using the adder rather than a separate comparator. You will see this in the ALU design.

Decoders

3.1 Definition

From the slides:

Decoder N inputs and 2^N possible outputs. Exactly one of the outputs is 1 and all the rest are 0s. The one output that is logically 1 is the output corresponding to the input pattern that the logic circuit is expected to detect. Circuit: inputs A, B with inverters; four AND gates outputting “1 if A, B is 00”, “01”, “10”, “11”. Example $A = 1, B = 0$: outputs 0, 0, 1, 0. (Handwritten: the AND gates are labelled with $\overline{A}\overline{B}$ etc.; outputs labelled EVEN, ODD, EVEN, ODD; “input pattern”, “one-hot”.)



A	B	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Formula : $n:2^n$ decoder

$$Y_i = m_i(x_{n-1}, \dots, x_0) \quad (i = 0, \dots, 2^n - 1)$$

Each output is one minterm of the inputs. Cost (direct design): n inverters and 2^n AND gates with n inputs each.

Why exactly one output is 1. Each input pattern is exactly one row of the truth table, and each minterm is 1 in exactly one row (Week 2). So the decoder's outputs are **one-hot**: a single 1 marks which pattern arrived.

Intuition : a decoder is a row selector

Think of the truth table of n inputs as 2^n numbered rows. A decoder is a machine that lights up the lamp next to the current row. It *decodes* a compact binary number (n wires) into an explicit “which one” (2^n wires).

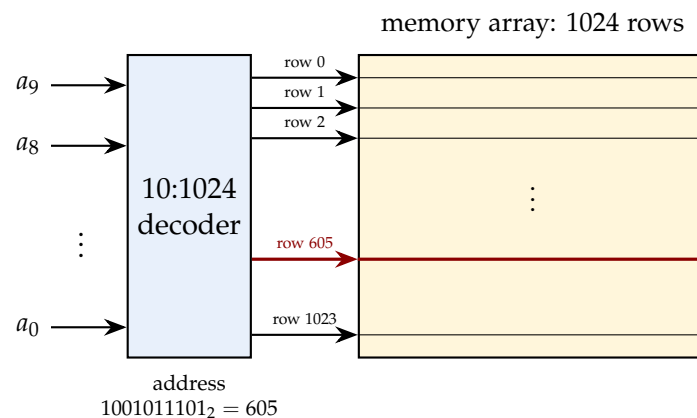
3.2 What decoders are used for

From the slides:

Decoder (uses) The decoder is useful in determining how to interpret a bit pattern. It could be the address of a row in DRAM that the processor intends to read from. It could be an instruction in the program and the processor has to decide what action to do! (based on instruction opcode). Example: $A = 1, B = 0$ activates the third output. (Handwritten: “address decoder”, hex digits $0 \dots 9, A, \dots, F$; opcodes $00 = \text{ADD}$, $01 = \text{SUB}$, $10 = \text{MUL}$, $11 = \text{DIV}$ with outputs labelled ADD cnt , SUB cnt , MUL cnt , DIV cnt .)

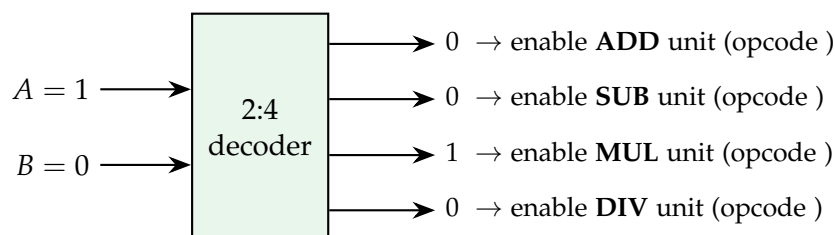
3.2.1 Address decoding

A memory with 2^n rows needs n address bits (Week 1: $n = \lceil \log_2 N \rceil$). A decoder turns the address into a single “word line” that enables one row.



The handwritten “10-bit $\rightarrow$ 1024 memory locations”: a 10-bit address selects exactly one of 2^{10} rows.

3.2.2 Instruction (opcode) decoding



The processor reads the opcode bits of an instruction; the decoder turns them into “do exactly this one operation”. With $AB = 10$, only the MUL unit is enabled.

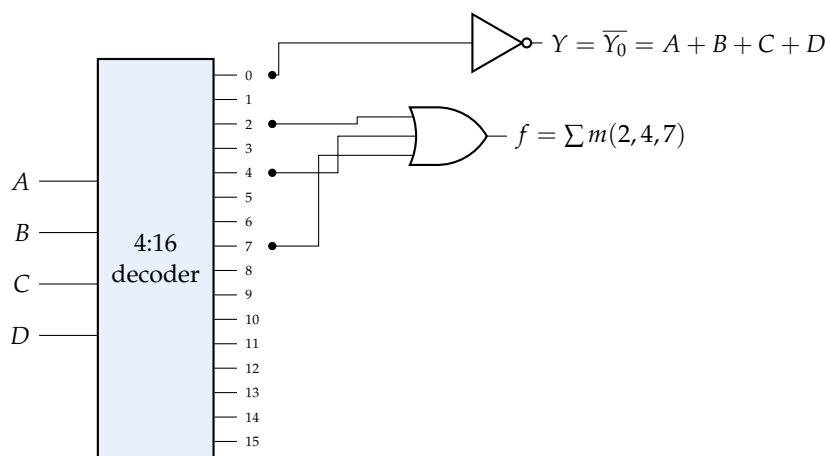
3.3 A decoder can implement any function

From the slides:

Decoder (function implementation) An $n:2^n$ decoder can be used to implement any function of n variables: connect variables to inputs; appropriate minterms summed using extra gates to form the function. A 4:16 decoder with inputs A, B, C, D on $I_3 \dots I_0$ and outputs 0–15 labelled $\overline{A}\overline{B}\overline{C}\overline{D} \dots ABCD$. **Exercise:** $Y = A + B + C + D$. Use the minterms as required! (Handwritten: “ $f_{SOP} \rightarrow \sum m(7, 4, 2)$ ” with outputs 7, 4, 2 circled; “at least one of $A/B/C/D$ has to be 1”; “ $\overline{ABCD} = \dots A + B + C + D$ ”.)

Since the decoder outputs *are* the minterms, any canonical SOP is just an OR of some decoder outputs:

$$F = \sum m(S) \implies F = \text{OR of decoder outputs } Y_i, i \in S.$$



Worked Example : the slide's exercise, $Y = A + B + C + D$

$Y = 0$ only when $A = B = C = D = 0$, i.e. only for minterm m_0 . So $Y = \sum m(1, 2, \dots, 15)$: a 15-input OR of outputs 1–15. Much cheaper: $Y = \overline{m_0}$, a single inverter on output 0 (handwritten: “at least one of $A/B/C/D$ has to be 1”). Proof: $\overline{m_0} = \overline{\overline{A}\overline{B}\overline{C}\overline{D}} = A + B + C + D$ by De Morgan.

Worked Example : handwritten $f = \sum m(2, 4, 7)$

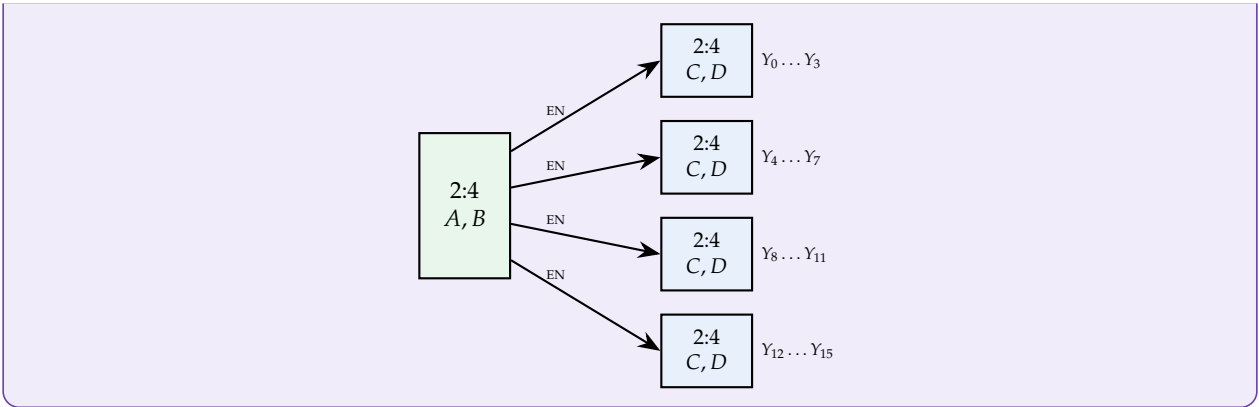
OR together decoder outputs 2, 4 and 7: a 3-input OR, as drawn above. On a 4:16 decoder these are $\overline{A}\overline{B}C\overline{D}$, $\overline{A}B\overline{C}\overline{D}$, $\overline{A}BCD$.

Key Idea

Decoder + OR gate = any SOP. If the function has many 1s, use a NOR on the 0 outputs instead (that is $\sum m(S') = \sum m(S)$), just as POS is shorter when 0s are few.

Beyond the Slides : enable inputs and decoder trees

Real decoders have an *enable* input EN : when $EN = 0$ all outputs are 0; when $EN = 1$ it decodes normally ($Y_i = EN \cdot m_i$). With enables, a big decoder can be built from small ones: a 4:16 decoder = one 2:4 decoder on A, B whose four outputs enable four 2:4 decoders on C, D . Output Y_{4j+k} is active when the first decoder selects j and the j -th second-level decoder selects k . Memory chips use exactly this two-level (row/column) decoding.



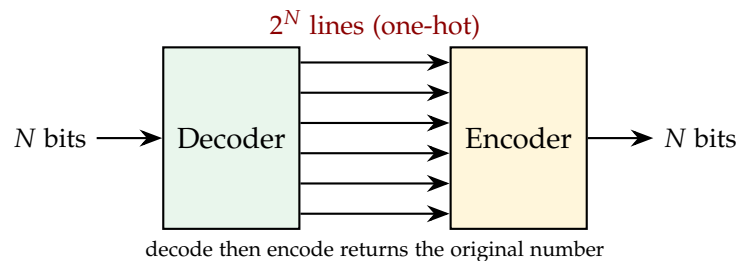
Chapter 4

Encoders and Priority Encoders

4.1 Encoder

From the slides:

Encoder Opposite of decoders. Binary encoders: 2^N inputs and N outputs. (N bits $\rightarrow$ Decoder $\rightarrow 2^N$ lines $\rightarrow$ Encoder $\rightarrow N$ bits.) **4-to-2 Encoder:** inputs I_3, I_2, I_1, I_0 ; outputs Y_1, Y_0 ; truth table rows 0001 $\rightarrow$ 00, 0010 $\rightarrow$ 01, 0100 $\rightarrow$ 10, 1000 $\rightarrow$ 11; circuit: $Y_1 = \text{OR}(I_3, I_2)$, $Y_0 = \text{OR}(I_3, I_1)$. (Handwritten: “ $2^3 = 8$ ”, 3-bit example; “input is encoded (only one input is anticipated)”.)



I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

$$I_3 \text{ --- } \text{OR} \text{ --- } Y_1 = I_3 + I_2$$

$$I_3 \text{ --- } \text{OR} \text{ --- } Y_0 = I_3 + I_1$$

(I_0 is not needed)

4.1.1 Deriving the equations

The encoder outputs the *index* of the active input in binary: $I_k = 1 \Rightarrow (Y_1 Y_0) = k$.

- Y_1 is the 2s bit of the index: it is 1 for $k = 2$ or $k = 3$. So $Y_1 = I_2 + I_3$.
- Y_0 is the 1s bit: 1 for $k = 1$ or $k = 3$. So $Y_0 = I_1 + I_3$.

In general, output bit j of a $2^N:N$ encoder is the OR of all inputs I_k whose index k has bit j set.

Formula : $2^N:N$ encoder

$$Y_j = \sum_{k: \text{bit } j \text{ of } k=1} I_k \quad [\text{OR of } 2^{N-1} \text{ inputs}], \quad j = 0, \dots, N-1.$$

Cost: N OR gates, each with 2^{N-1} inputs (input I_0 appears in none).

Worked Example : 8-to-3 encoder (handwritten $2^3 = 8$)

$Y_2 = I_4 + I_5 + I_6 + I_7$; $Y_1 = I_2 + I_3 + I_6 + I_7$; $Y_0 = I_1 + I_3 + I_5 + I_7$. Input I_6 active $\Rightarrow Y_2 Y_1 Y_0 = 110 = 6$.

4.1.2 The hidden assumption: exactly one input is active

The simple OR equations are only correct if the inputs are one-hot (“only one input is anticipated”). The 4-row table lists 4 of the $2^4 = 16$ input combinations; the circuit’s output on the other 12 is whatever the ORs happen to give:

- $I_2 = I_1 = 1$: $Y_1 Y_0 = 11$ — claims input 3 is active, which is wrong.
- All inputs 0: $Y_1 Y_0 = 00$ — indistinguishable from “ I_0 is active”.

The priority encoder fixes both problems.

4.2 Priority encoder**From the slides:**

Priority Encoder Highest numbered inputs have priority when multiple inputs are asserted at the same time. **Don’t Care Condition.** Example: 4-to-2 priority encoder with outputs Y_1 , Y_0 , None. Table: $1\ X\ X\ X \rightarrow 1\ 1\ 0$; $0\ 1\ X\ X \rightarrow 1\ 0\ 0$; $0\ 0\ 1\ X \rightarrow 0\ 1\ 0$; $0\ 0\ 0\ 1 \rightarrow 0\ 0\ 0$; $0\ 0\ 0\ 0 \rightarrow 0\ 0\ 1$. $Y_1 = I_3 + \bar{I}_3 I_2 = I_3 + I_2$; $Y_0 = I_3 + \bar{I}_3 \bar{I}_2 I_1 = I_3 + \bar{I}_2 I_1$. (Handwritten: “CKT behaviour is AGNOSTIC to” the X inputs.)

I_3	I_2	I_1	I_0	Y_1	Y_0	None
1	X	X	X	1	1	0
0	1	X	X	1	0	0
0	0	1	X	0	1	0
0	0	0	1	0	0	0
0	0	0	0	0	0	1

4.2.1 Don’t-care conditions in the inputs

Here X in an *input* column means “this row applies whatever the value”: row 1 stands for all 8 patterns with $I_3 = 1$. Once I_3 is active, the circuit is “agnostic” to lower inputs. The 5 compressed rows cover all 16 input combinations ($8 + 4 + 2 + 1 + 1 = 16$). (This is different from the output don’t-cares of Week 3, which leave the designer free to choose an output.)

$I_3 = 1$: 8 rows	01 X X: 4	001 X: 2	0001	0000
--------------------	-----------	----------	------	------

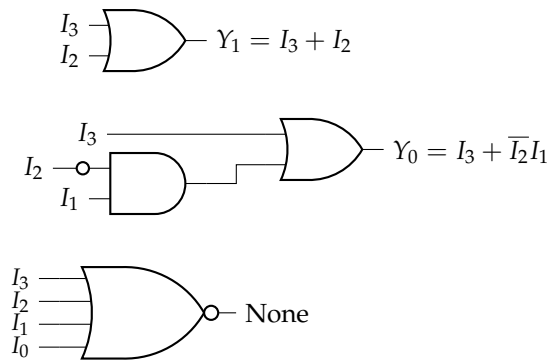
the 16 input combinations, grouped by the highest active input

4.2.2 Deriving the equations

Y_1 is 1 in rows 1 and 2: $Y_1 = I_3 + \bar{I}_3 I_2$. By T11D ($X + \bar{X}Y = X + Y$): $Y_1 = I_3 + I_2$.

Y_0 is 1 in rows 1 and 3: $Y_0 = I_3 + \bar{I}_3 \bar{I}_2 I_1$. By T11D with $X = I_3$: $Y_0 = I_3 + \bar{I}_2 I_1$.

None is 1 only in the last row: $\text{None} = \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0 = \overline{I_3 + I_2 + I_1 + I_0}$ (a 4-input NOR).

**Worked Example : two requests at once**

$I_2 = I_1 = 1$, others 0. Row 2 applies (highest active is I_2): $Y_1 = 0 + 1 = 1$, $Y_0 = 0 + \overline{1} \cdot 1 = 0$: output $10 = 2$. Correct — input 2 wins. (The plain encoder would have output 11.)

Intuition : interrupts

Several devices (keyboard, disk, network, timer) may request the processor's attention at once. A priority encoder tells the processor the number of the most urgent request, and "None" tells it there is no request at all. That is exactly the job of an interrupt controller.

Common Pitfall

Without the None output, "no input active" and " I_0 active" both give $Y_1Y_0 = 00$. Whenever an encoder's all-zero input is possible, you need a separate valid/None signal.

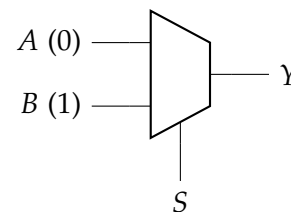
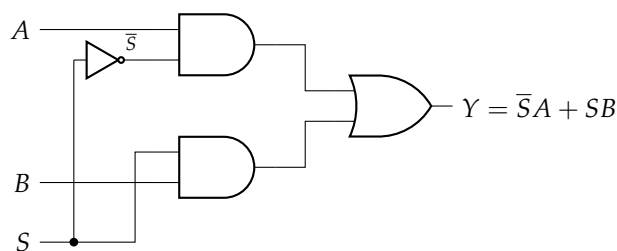
Multiplexers

5.1 Definition

From the slides:

Multiplexer Selects one of the N inputs to connect it to the output. Needs a $\log_2 N$ -bit control input. **2:1 MUX**: inputs A, B , select S ; two AND gates (one gated by $\bar{S}$, one by S) feeding an OR; output C . (Handwritten: “4:1 mux, 8:1 mux”; “ $N = \#$ of inputs, select line, $\log_2 N$ ”; “ $S = 0 \Rightarrow A, S = 1 \Rightarrow B$ ”; a second drawing with $S = 0$ shows A passing through and the B path OFF.)

A multiplexer (“mux”) is a digitally controlled switch: the select input says *which* data input is copied to the output.



S	Y
0	A
1	B

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Canonical SOP of the full table: $Y = \sum m(2, 3, 5, 7)$ over (S, A, B) :

$$Y = \bar{S}A\bar{B} + \bar{S}AB + S\bar{A}B + SAB = \bar{S}A + SB.$$

5.1.1 How it works

- $S = 0$: the top AND sees $\bar{S} = 1$, so it passes A ; the bottom AND sees $S = 0$ and outputs 0 (“OFF”). $Y = A + 0 = A$.
- $S = 1$: the top AND is OFF; the bottom passes B . $Y = 0 + B = B$.

Each AND gate acts as a gate in the everyday sense — open or shut — controlled by the select line. This is the handwritten “ $S = 0 \Rightarrow A$, OFF” drawing.

5.2 Bigger multiplexers

Formula : $N:1$ multiplexer

$$\text{select bits} = \log_2 N, \quad Y = \sum_{i=0}^{N-1} m_i(S_{k-1}, \dots, S_0) \cdot D_i \quad (k = \log_2 N)$$

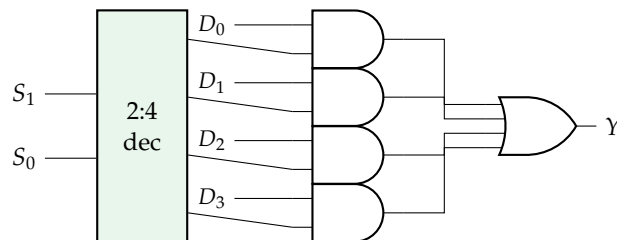
Each data input is ANDed with the minterm of the select lines that picks it.

Why $\log_2 N$. The select value must name one of N inputs; k bits name 2^k things, so $2^k = N$, $k = \log_2 N$ (Week 1).

Worked Example : 4:1 multiplexer

$$Y = \overline{S_1} \overline{S_0} D_0 + \overline{S_1} S_0 D_1 + S_1 \overline{S_0} D_2 + S_1 S_0 D_3.$$

The four select minterms are exactly the outputs of a 2:4 decoder: a mux is a decoder whose outputs gate the data, followed by an OR.

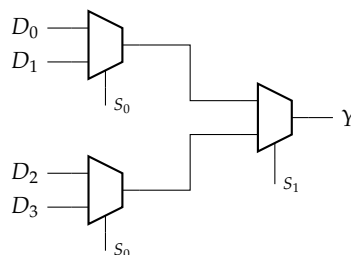


The D_i inputs are drawn entering the upper pin of each AND for clarity; the decoder output enables exactly one AND.

Mux	data inputs N	select bits $\log_2 N$	AND gates (direct design)
2:1	2	1	2
4:1	4	2	4 (3-input)
8:1	8	3	8 (4-input)
16:1	16	4	16 (5-input)

5.2.1 Mux trees

Larger muxes can be built from 2:1 muxes: a 4:1 mux is two 2:1 muxes controlled by S_0 (choosing D_0/D_1 and D_2/D_3), feeding a third 2:1 mux controlled by S_1 . In general an $N:1$ mux needs $N - 1$ two-to-one muxes arranged in $\log_2 N$ levels.



5.3 A multiplexer can implement any function

Beyond the Slides : mux as a lookup table

Connect the n variables of F to the select inputs of a $2^n:1$ mux and tie data input D_i to the constant $F(\text{row } i)$. Then $Y = \sum_i m_i D_i = \sum_{i:F_i=1} m_i = F$. The mux literally looks up the truth table. This is how FPGAs implement logic: small lookup tables (LUTs) are multiplexers fed by memory bits.

Example: majority $M(A, B, C) = \sum m(3, 5, 6, 7)$ with an 8:1 mux: $D_3 = D_5 = D_6 = D_7 = 1$, the rest 0.

Halving the mux (Shannon expansion). $F = \bar{C}F(C=0) + CF(C=1)$. Use A, B as selects of a 4:1 mux and feed each D_i with 0, 1, C or $\bar{C}$. For the majority: rows $AB = 00 \rightarrow 0$; $01 \rightarrow C$; $10 \rightarrow C$; $11 \rightarrow 1$. So a 4:1 mux with $D = (0, C, C, 1)$ computes M .

Beyond the Slides : demultiplexer

The reverse block, a *demultiplexer*, routes one input to one of N outputs chosen by $\log_2 N$ select bits. It is a decoder with an enable: the data input drives the enable.

Key Idea

Decoder: n bits in $\rightarrow$ one of 2^n lines active. Encoder: the reverse. Multiplexer: $\log_2 N$ select bits choose one of N data inputs. All three are built from the same minterm idea: **each select pattern is a minterm, and a minterm is 1 in exactly one row.**

Chapter 6

Beyond the Slides: Full Adders and PLAs

The standard-blocks slide lists *full adders* and *PLAs*, but the Week 4 deck does not develop them. This chapter gives a short, textbook-standard introduction (Mano, Ch. 4; Harris & Harris, Ch. 5) because the ALU in Module 2 is built on adders. Check with your instructor how much of it is examinable.

6.1 Half adder (recap from Chapter 2)

$s = a \oplus b$, $c = ab$. It adds two bits but has no input for a carry coming from a lower position.

6.2 Full adder

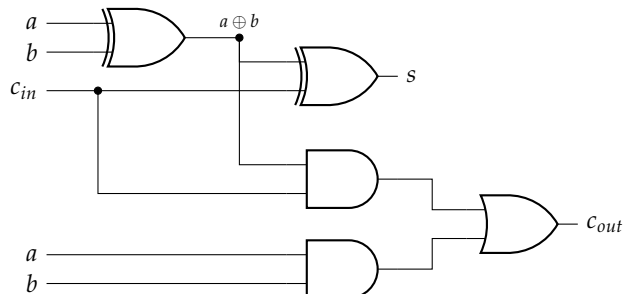
Adds three bits: a , b and a carry-in c_{in} , producing sum s and carry-out c_{out} , with $a + b + c_{in} = 2c_{out} + s$.

a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$s = \sum m(1, 2, 4, 7) = a \oplus b \oplus c_{in}$
(odd parity — the three-switch function from Week 3!)

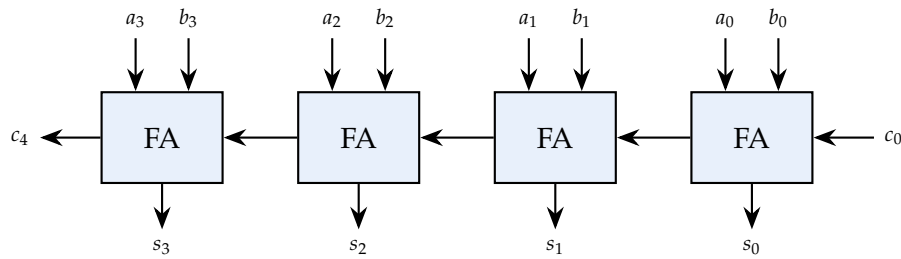
$c_{out} = \sum m(3, 5, 6, 7) = ab + ac_{in} + bc_{in}$
(the majority function) $= ab + c_{in}(a \oplus b)$

Derivation of the second form of c_{out} . $ab + ac_{in} + bc_{in} = ab + c_{in}(a + b)$. When $a = b = 1$ the ab term already gives 1, so inside $c_{in}(\cdot)$ we only need the case where exactly one of a, b is 1: $a + b$ can be replaced by $a \oplus b$. This lets the full adder re-use the XOR already computed for s .



6.2.1 Ripple-carry adder

Chain N full adders, each carry-out feeding the next carry-in, to add two N -bit numbers:



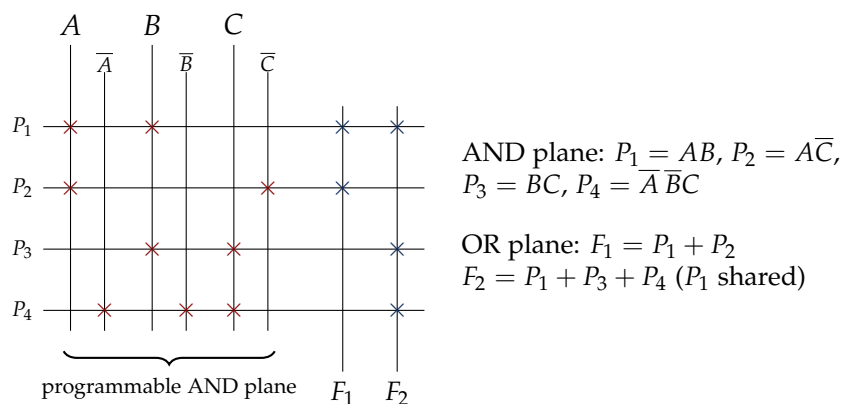
Its delay grows with N because each carry must “ripple” through all lower stages — a multilevel circuit with a long critical path (Week 3). Faster adders (carry-lookahead) trade more gates for fewer levels.

6.3 Programmable Logic Arrays (PLAs)

The slide’s handwritten note: “re-configurable, programmable hardware logic”. A PLA is a two-level AND–OR structure whose connections are *programmable*:

- an **AND plane**: each AND gate can be connected to any input literal (x_i or $\bar{x}_i$), producing chosen product terms;
- an **OR plane**: each output ORs any chosen subset of those product terms.

So a PLA implements several SOP functions at once, and product terms can be *shared* between outputs.



A $\times$ marks a programmed connection. Modern descendants (CPLDs, FPGAs) use the same idea of programmable connections or the multiplexer lookup tables of Chapter 5.

Part II

Practice

Chapter 7

Numerical Problems with Worked Solutions

7.1 XOR and comparators

Problem 1 *XOR / XNOR evaluation*

Evaluate $1 \oplus 0$, $1 \oplus 1$, $\overline{0 \oplus 0}$, and $1 \oplus 1 \oplus 1$.

Solution

$1 \oplus 0 = 1$; $1 \oplus 1 = 0$; $\overline{0 \oplus 0} = \bar{0} = 1$; $1 \oplus 1 \oplus 1 = 0 \oplus 1 = 1$ (odd number of 1s).

Problem 2 *Controlled inverter*

Show $x \oplus 1 = \bar{x}$ and $x \oplus 0 = x$ using $x \oplus y = x\bar{y} + \bar{x}y$.

Solution

$x \oplus 1 = x \cdot 0 + \bar{x} \cdot 1 = \bar{x}$. $x \oplus 0 = x \cdot 1 + \bar{x} \cdot 0 = x$.

Problem 3 *Equality comparator cost*

How many 2-input gates are needed for an 8-bit equality comparator ($A \neq B$ output) built as in the slides? How many gate levels?

Solution

8 XORs (one per bit) + an OR tree combining 8 signals: $8 - 1 = 7$ two-input ORs. Total 15 gates. Levels: 1 (XOR) + $\log_2 8 = 3$ (OR tree) = 4.

Problem 4 *Where do they differ?*

Compare $A = 1101$ and $B = 1100$.

Solution

$d_3d_2d_1d_0 = 0001$: they differ in bit 0 only. $A \neq B = 1$; $A = B$ is 0.

Problem 5 *Equal numbers*

Compare $A = 0110$ and $B = 0110$.

Solution

Every $d_i = 0$, so $A \neq B = 0$, $E = 1$.

Problem 6 *Magnitude, $A > B$*

$A = 0110$ (6), $B = 0101$ (5). Evaluate G using the magnitude-comparator formula.

Solution

Bit 3: 0,0 ($e_3 = 1$, term $A_3\bar{B}_3 = 0$). Bit 2: 1,1 ($e_2 = 1$, term 0). Bit 1: $A_1 = 1, B_1 = 0$: term $e_3e_2A_1\bar{B}_1 = 1$. So $G = 1$. ✓ $6 > 5$.

Problem 7 *Magnitude, $A < B$*

$A = 1001$ (9), $B = 1100$ (12). Which of G, L, E is 1, and which bit decides?

Solution

Bit 3 equal (1,1). Bit 2: $A_2 = 0, B_2 = 1$: the term $e_3\bar{A}_2B_2 = 1$ of L . $L = 1$, decided by bit 2.

Problem 8 *2-bit comparator counts*

For 2-bit A and B , how many of the 16 input combinations give $A > B$, $A < B$, $A = B$? Check with the formula for G at $A = 10$, $B = 01$.

Solution

Equal: 4 (the diagonal). The remaining 12 split evenly by symmetry: 6 each. $G = a_1\bar{b}_1 + e_1a_0\bar{b}_0 = 1 \cdot 1 + 0 = 1$ ✓ ($2 > 1$).

7.2 Decoders

Problem 9 *Decoder sizes*

For a 3:8 and a 5:32 decoder built directly, how many AND gates (and with how many inputs), and how many inverters?

Solution

3:8: 8 AND3, 3 inverters. 5:32: 32 AND5, 5 inverters. In general 2^n AND gates of n inputs.

Problem 10 *Memory address decoder*

A memory has 4096 rows. What decoder selects a row?

Solution

$4096 = 2^{12}$: 12 address bits, a 12:4096 decoder (in practice built as a tree).

Problem 11 *Active output*

A 3:8 decoder receives $ABC = 101$. Which output is 1?

Solution

$101_2 = 5$: $Y_5 = A\bar{B}C = 1$; all others 0.

Problem 12 *Opcode decoding*

With opcodes 00 = ADD, 01 = SUB, 10 = MUL, 11 = DIV on inputs A (MSB), B , which unit is enabled for $AB = 01$?

Solution

01 = 1: output Y_1 = SUB.

Problem 13 *Function from a decoder*

Implement $F(A, B, C) = \sum m(0, 3, 5, 6)$ with a 3:8 decoder.

Solution

$F = Y_0 + Y_3 + Y_5 + Y_6$: one 4-input OR on those outputs.

Problem 14 *Many ones*

Implement $F(A, B, C) = \sum m(0, 1, 2, 3, 4, 5, 6)$ cheaply with a 3:8 decoder.

Solution

F is 0 only at m_7 , so $F = \overline{Y_7}$: one inverter ($= \overline{ABC}$, a 3-input NAND). A 7-input OR also works but costs more.

Problem 15 *The slide's exercise*

With a 4:16 decoder, implement $Y = A + B + C + D$.

Solution

$Y = \sum m(1, \dots, 15) = \overline{Y_0}$: invert output 0. By De Morgan $\overline{\overline{A}\overline{B}\overline{C}\overline{D}} = A + B + C + D$.

Problem 16 *Full adder from a decoder*

Implement a full adder with a 3:8 decoder (inputs a, b, c_{in}) and OR gates.

Solution

$s = \sum m(1, 2, 4, 7) = Y_1 + Y_2 + Y_4 + Y_7$; $c_{out} = \sum m(3, 5, 6, 7) = Y_3 + Y_5 + Y_6 + Y_7$. Two 4-input ORs share one decoder.

Problem 17 *Decoder tree*

Build a 5:32 decoder from 3:8 decoders with enables and one 2:4 decoder. How many decoders?

Solution

The 2 high bits feed a 2:4 decoder; its 4 outputs enable four 3:8 decoders driven by the 3 low bits. $4 \times 8 = 32$ outputs; 5 decoders in total.

7.3 Encoders

Problem 18 *8-to-3 encoder*

In an 8:3 encoder, $I_5 = 1$ (others 0). Find $Y_2Y_1Y_0$ from the OR equations.

Solution

$Y_2 = I_4 + I_5 + I_6 + I_7 = 1$; $Y_1 = I_2 + I_3 + I_6 + I_7 = 0$; $Y_0 = I_1 + I_3 + I_5 + I_7 = 1$. Output 101 = 5 ✓.

Problem 19 *Encoder misbehaviour*

The same 8:3 encoder gets $I_2 = I_4 = 1$. What does it output?

Solution

$Y_2 = 1$ (from I_4), $Y_1 = 1$ (from I_2), $Y_0 = 0$: 110 = 6 — neither 2 nor 4. The plain encoder assumes one-hot input.

Problem 20 *Encoder cost*

How many OR gates, and with how many inputs, does a 16:4 encoder need?

Solution

4 outputs; each is the OR of the $2^{4-1} = 8$ inputs whose index has that bit set: 4 OR8 gates.

Problem 21 *Priority encoder evaluation*

4:2 priority encoder: find Y_1Y_0 and None for (a) 0111, (b) 0000, (c) 1010 (inputs $I_3I_2I_1I_0$).

Solution

(a) Highest active is I_2 : $Y_1 = I_3 + I_2 = 1$, $Y_0 = I_3 + \overline{I_2}I_1 = 0 + 0 = 0$: 10, None = 0. (b) $Y_1Y_0 = 00$, None = 1. (c) $I_3 = 1$: 11, None = 0.

Problem 22 *8:3 priority encoder*

Derive Y_2, Y_1, Y_0 for an 8:3 priority encoder (highest index wins).

Solution

Output k = highest active index. $Y_2 = 1$ iff $k \geq 4$: $Y_2 = I_7 + I_6 + I_5 + I_4$. $Y_1 = 1$ iff $k \in \{2, 3, 6, 7\}$: $Y_1 = I_7 + I_6 + \overline{I_5}\overline{I_4}(I_3 + I_2)$ (for $k = 2, 3$ nothing above may be active; the $\overline{I_7}\overline{I_6}$ factors drop out by T11D). $Y_0 = 1$ iff $k \in \{1, 3, 5, 7\}$: $Y_0 = I_7 + \overline{I_6}I_5 + \overline{I_6}\overline{I_4}I_3 + \overline{I_6}\overline{I_4}\overline{I_2}I_1$.

7.4 Multiplexers

Problem 23 *Select lines*

How many select bits for a 32:1 and a 64:1 mux?

Solution

$$\log_2 32 = 5; \log_2 64 = 6.$$

Problem 24 *2:1 mux evaluation*

$$Y = \bar{S}A + SB \text{ with } S = 1, A = 0, B = 1.$$

Solution

$$Y = 0 \cdot 0 + 1 \cdot 1 = 1 \text{ (= } B, \text{ as } S = 1 \text{ selects } B).$$

Problem 25 *4:1 mux evaluation*

Data $(D_0, D_1, D_2, D_3) = (0, 1, 1, 0)$, $S_1 S_0 = 10$. Find Y .

Solution

$$S_1 S_0 = 10 = 2: Y = D_2 = 1.$$

Problem 26 *Mux tree*

How many 2:1 muxes and levels make a 16:1 mux?

Solution

$$N - 1 = 15 \text{ muxes in } \log_2 16 = 4 \text{ levels } (8 + 4 + 2 + 1).$$

Problem 27 *XOR from a mux*

Implement $A \oplus B$ with a 4:1 mux ($S_1 = A, S_0 = B$).

Solution

Truth table outputs for $AB = 00, 01, 10, 11$: 0, 1, 1, 0. So $(D_0, D_1, D_2, D_3) = (0, 1, 1, 0)$.

Problem 28 *Odd parity with a 4:1 mux*

Implement $F(A, B, C) = \sum m(1, 2, 4, 7)$ with a 4:1 mux, A, B on the selects.

Solution

$AB = 00$: rows 0,1 have $F = 0, 1 \Rightarrow D_0 = C$. $AB = 01$: rows 2,3: $1, 0 \Rightarrow D_1 = \bar{C}$. $AB = 10$: rows 4,5: $1, 0 \Rightarrow D_2 = \bar{C}$. $AB = 11$: rows 6,7: $0, 1 \Rightarrow D_3 = C$. $D = (C, \bar{C}, \bar{C}, C)$.

7.5 Adders and PLAs (beyond)

Problem 29 *Full adder evaluation*

Find s and c_{out} for $a = 1, b = 1, c_{in} = 1$, and for $a = 1, b = 0, c_{in} = 1$.

Solution

$1 + 1 + 1 = 3 = 11_2$: $c_{out} = 1, s = 1$. $1 + 0 + 1 = 2 = 10_2$: $c_{out} = 1, s = 0$.

Problem 30 *Ripple-carry addition*

Add $0110 + 0111$ with a 4-bit ripple-carry adder ($c_0 = 0$). Give each carry.

Solution

Bit 0: $0 + 1 + 0 = 1$: $s_0 = 1, c_1 = 0$. Bit 1: $1 + 1 + 0 = 2$: $s_1 = 0, c_2 = 1$. Bit 2: $1 + 1 + 1 = 3$: $s_2 = 1, c_3 = 1$. Bit 3: $0 + 0 + 1 = 1$: $s_3 = 1, c_4 = 0$. Sum $1101 = 13 = 6 + 7 \checkmark$.

Problem 31 *Ripple delay*

Each full adder's carry path has 2 gate delays of 50 ps. Estimate the worst-case carry delay of a 32-bit ripple-carry adder.

Solution

The carry may pass through all 32 stages: $32 \times 2 \times 50 = 3\,200 \text{ ps} = 3.2 \text{ ns}$. This linear growth is why fast adders use carry lookahead.

Problem 32 *PLA size*

A PLA has 3 inputs, 4 product terms and 2 outputs. How many programmable crosspoints are in the AND plane and OR plane?

Solution

AND plane: each of 4 product lines can connect to $2 \times 3 = 6$ literal lines: 24. OR plane: $4 \times 2 = 8$. Total 32.

Chapter 8

Subjective Questions with Answers

Q1. What are combinational building blocks and why are they used?

Answer

Standard circuits (tens to hundreds of transistors, historically MSI chips) that perform a common function — comparing, decoding, encoding, selecting, adding. They hide gate-level detail behind a functional description, so larger systems can be designed with blocks instead of individual gates, like Lego bricks. Each block is designed and verified once and reused.

Q2. Define XOR and XNOR, and explain why XOR is described as “similar to OR”.

Answer

XOR $x \oplus y = x\bar{y} + \bar{x}y$ is 1 when inputs differ; XNOR $= xy + \bar{x}\bar{y}$ is 1 when inputs are equal. XOR matches OR on the first three rows and differs only when both inputs are 1 (OR gives 1, XOR gives 0).

Q3. Explain how XOR relates to binary addition.

Answer

Adding two bits gives a 2-bit result: the sum bit is 1 when exactly one input is 1 (XOR) and the carry is 1 when both are 1 (AND). So a half adder is one XOR and one AND — the first step towards an adder.

Q4. Design a 1-bit and an N -bit equality comparator.

Answer

1-bit: $E = \overline{x \oplus y}$ (one XNOR). N -bit: compute $d_i = A_i \oplus B_i$ for each bit; $A \neq B = \sum d_i$ (OR of XORs, as on the slide), and $A = B = \overline{\sum d_i} = \prod \bar{d}_i$. Cost: N XORs and an N -input OR (or $N - 1$ two-input ORs in a tree).

Q5. Why is a comparator designed structurally rather than from a truth table?

Answer

An N -bit comparator has $2N$ inputs, so its truth table has 2^{2N} rows (256 for 4 bits, 2^{64} for 32 bits). Building it from per-bit blocks and a combining tree is compact, regular, and scales linearly with N .

Q6. Derive the logic for $A > B$ for N -bit numbers.

Answer

Scan from the MSB. Let k be the highest position where the bits differ; all higher bits are equal ($e_i = 1$) and bit k decides because 2^k exceeds the sum of all lower weights. $G = \sum_k (\prod_{i>k} e_i) A_k \overline{B}_k$, with $e_i = A_i \oplus B_i$. L uses $\overline{A}_k B_k$; $E = \prod e_i$.

Q7. Define a decoder and explain why exactly one output is active.

Answer

An $n:2^n$ decoder has n inputs and 2^n outputs; output Y_i is minterm m_i of the inputs. Each input pattern is exactly one row, and each minterm is 1 in exactly one row, so exactly one output is 1 (one-hot).

Q8. Give two uses of decoders in a computer.

Answer

(1) Address decoding: an n -bit address selects one of 2^n memory rows (10 bits $\rightarrow$ 1024 locations). (2) Instruction decoding: opcode bits select which operation unit (ADD, SUB, MUL, DIV) is enabled.

Q9. How can a decoder implement any Boolean function? Illustrate with the slide exercise.

Answer

The decoder outputs are all the minterms, so any canonical SOP $\sum m(S)$ is the OR of outputs Y_i , $i \in S$. For $Y = A + B + C + D$, $Y = \sum m(1..15)$; more cheaply $Y = \overline{Y}_0$, since $\overline{ABCD} = A + B + C + D$.

Q10. What is an encoder? Derive the 4-to-2 encoder equations.

Answer

The inverse of a decoder: 2^N inputs (one active) and N outputs giving the active input's index in binary. Y_1 (2s bit) is 1 for inputs 2, 3: $Y_1 = I_2 + I_3$. Y_0 (1s bit) for inputs 1, 3: $Y_0 = I_1 + I_3$.

Q11. What goes wrong with a simple encoder, and how does a priority encoder fix it?

Answer

The OR equations assume exactly one active input. With two active inputs the outputs combine into a wrong code (e.g. $I_1, I_2 \rightarrow 11$), and with none the output 00 looks like I_0 . A priority encoder outputs the index of the highest-numbered active input and adds a None (valid) output for the all-zero case.

Q12. Derive the priority encoder equations from the slide's table.

Answer

Y_1 is 1 when $I_3 = 1$, or $I_3 = 0$ and $I_2 = 1$: $Y_1 = I_3 + \overline{I_3}I_2 = I_3 + I_2$ (T11D). Y_0 is 1 when $I_3 = 1$, or $I_3 = I_2 = 0$ and $I_1 = 1$: $Y_0 = I_3 + \overline{I_3}\overline{I_2}I_1 = I_3 + \overline{I_2}I_1$. None = $\overline{I_3} + \overline{I_2} + \overline{I_1} + \overline{I_0}$.

Q13. Explain the meaning of X in the priority encoder's input columns and how it differs from an output don't-care.

Answer

An input X means the row applies for either value — the circuit is agnostic to that input once a higher-priority input is active; a row with k X's stands for 2^k rows. An output don't-care (Week 3) means the designer may choose the output value for that input combination to simplify the logic.

Q14. Define a multiplexer and derive the 2:1 mux equation.

Answer

A mux copies one of N data inputs to its output, chosen by $\log_2 N$ select bits. 2:1: $Y = A$ when $S = 0$, B when $S = 1$. Truth table ones: $\overline{S}A\overline{B}, \overline{S}AB, S\overline{A}\overline{B}, SAB$, which simplify to $Y = \overline{S}A + SB$: each AND gate passes its input only when selected.

Q15. Why does an N :1 mux need $\log_2 N$ select lines, and how is a mux related to a decoder?

Answer

The select value must name one of N inputs; k bits name 2^k values, so $k = \log_2 N$. Internally the select lines are decoded into minterms (a decoder), each ANDed with one data input, and the results ORed: $Y = \sum m_i(S)D_i$.

Q16. Explain how a multiplexer can implement any Boolean function.

Answer

Put the function's variables on the select lines of a 2^n :1 mux and tie each D_i to the output value of row i ; the mux then looks up the truth table. Using Shannon expansion, a 2^{n-1} :1 mux suffices if each data input is 0, 1, x or $\overline{x}$ for the last variable. FPGAs implement logic this way (LUTs).

Q17. Compare decoder, encoder and multiplexer in one table-style answer.

Answer

Decoder: n inputs $\rightarrow 2^n$ one-hot outputs (binary $\rightarrow$ "which one"). Encoder: 2^n one-hot inputs $\rightarrow n$ outputs ("which one" $\rightarrow$ binary). Mux: N data + $\log_2 N$ select $\rightarrow 1$ output (choose). All rely on minterms: each select/input pattern is 1 in exactly one row.

Q18. What are full adders and why are they building blocks? (beyond)

Answer

A full adder adds a , b and a carry-in: $s = a \oplus b \oplus c_{in}$, $c_{out} = ab + c_{in}(a \oplus b)$. Chaining N of them (ripple carry) adds N -bit numbers, which is the arithmetic core of the ALU.

Q19. What is a PLA? (beyond)

Answer

A programmable logic array: a programmable AND plane generating chosen product terms from the input literals and a programmable OR plane combining them into several outputs. It implements multiple SOP functions, sharing product terms, and can be reconfigured — “programmable hardware logic”.

Q20. Walk through the design of “which of 4 devices should the CPU serve?” using this week’s blocks.

Answer

Requests $I_0..I_3$ feed a 4:2 priority encoder: Y_1Y_0 = number of the highest-priority request, None = no request. Y_1Y_0 can drive the select lines of a 4:1 mux choosing that device’s data, and a 2:4 decoder can turn Y_1Y_0 back into a one-hot “acknowledge” signal for the selected device.

Chapter 9

Multiple-Choice Questions

Answers and explanations follow in Section 9.2.

9.1 Questions

Blocks, XOR, comparators

1. Combinational building blocks with tens to hundreds of transistors used to be called:

- (a) SSI
- (b) MSI
- (c) LSI
- (d) VLSI

2. XOR differs from OR only when:

- (a) both inputs are 0
- (b) inputs differ
- (c) both inputs are 1
- (d) never

3. $x \oplus 1$ equals:

- (a) x
- (b) $\bar{x}$
- (c) 1
- (d) 0

4. A 1-bit equality comparator is a single:

- (a) AND
- (b) OR
- (c) XNOR
- (d) NAND

5. In a half adder, the sum and carry are:

- (a) XOR and AND
- (b) AND and OR
- (c) OR and XOR
- (d) XNOR and NOR

6. The slide's N -bit comparator outputs $A \neq B$ as:

- (a) AND of XORs

- (b) OR of XORs
 - (c) OR of XNORs
 - (d) NOR of ANDs
7. $A = B$ for N -bit numbers equals:
- (a) $\prod (A_i \oplus B_i)$
 - (b) $\prod \overline{A_i \oplus B_i}$
 - (c) $\sum \overline{A_i \oplus B_i}$
 - (d) $\sum A_i B_i$
8. Comparing 1011 and 1001, the bits differ at position:
- (a) 0
 - (b) 1
 - (c) 2
 - (d) 3
9. A 4-bit comparator's full truth table has:
- (a) 16 rows
 - (b) 64 rows
 - (c) 256 rows
 - (d) 8 rows
10. For 1-bit a, b , $a > b$ equals:
- (a) ab
 - (b) $a\bar{b}$
 - (c) $\bar{a}b$
 - (d) $a \oplus b$
11. In magnitude comparison, the deciding bit is:
- (a) the LSB
 - (b) the highest bit where A and B differ
 - (c) any differing bit
 - (d) the MSB always

Decoders

12. A decoder with N inputs has how many outputs?
- (a) N
 - (b) $2N$
 - (c) 2^N
 - (d) $\log_2 N$
13. In a decoder, for any input:
- (a) all outputs are 1

- (b) exactly one output is 1
- (c) at most N outputs are 1
- (d) outputs are random

14. Decoder output Y_i equals:

- (a) maxterm M_i
- (b) minterm m_i
- (c) input i
- (d) $\overline{m_i}$

15. For $A = 1$, $B = 0$, a 2:4 decoder activates:

- (a) Y_0
- (b) Y_1
- (c) Y_2
- (d) Y_3

16. A 10-bit address decoder selects one of:

- (a) 10 rows
- (b) 100 rows
- (c) 1024 rows
- (d) 2048 rows

17. With opcodes 00 ADD, 01 SUB, 10 MUL, 11 DIV, input 10 enables:

- (a) ADD
- (b) SUB
- (c) MUL
- (d) DIV

18. With a 4:16 decoder, $Y = A + B + C + D$ is most cheaply:

- (a) OR of all 16 outputs
- (b) $\overline{Y_0}$
- (c) Y_{15}
- (d) Y_0

19. A decoder plus an OR gate can implement:

- (a) only AND functions
- (b) any function of its inputs
- (c) only symmetric functions
- (d) only XOR

20. A 5:32 decoder built from 3:8 decoders and a 2:4 decoder uses how many decoders?

- (a) 4
- (b) 5
- (c) 8
- (d) 9

Encoders and priority encoders

21. An encoder has 2^N inputs and:

- (a) 2^N outputs
- (b) N outputs
- (c) 1 output
- (d) N^2 outputs

22. In a 4-to-2 encoder, $Y_1 =$

- (a) $I_1 + I_3$
- (b) $I_2 + I_3$
- (c) $I_0 + I_1$
- (d) I_3

23. Which input is not needed in a 4-to-2 encoder's equations?

- (a) I_0
- (b) I_1
- (c) I_2
- (d) I_3

24. A plain 4-to-2 encoder with $I_1 = I_2 = 1$ outputs:

- (a) 01
- (b) 10
- (c) 11
- (d) 00

25. In the priority encoder, $Y_0 =$

- (a) $I_3 + I_1$
- (b) $I_3 + \overline{I_2}I_1$
- (c) $I_2 + I_1$
- (d) $\overline{I_3}I_1$

26. The priority encoder's None output is:

- (a) I_0
- (b) $\overline{I_3 + I_2 + I_1 + I_0}$
- (c) $I_3I_2I_1I_0$
- (d) $Y_1 + Y_0$

27. Priority encoder input 0110 gives $Y_1Y_0 =$

- (a) 01
- (b) 10
- (c) 11
- (d) 00

28. The priority encoder row 1 X X X covers how many input combinations?

- (a) 1
- (b) 3
- (c) 8
- (d) 16

29. “CKT behaviour is agnostic to” the X inputs means:

- (a) the circuit ignores them in that row
- (b) they must be 0
- (c) they must be 1
- (d) the output is X

30. Priority encoders are used in:

- (a) memory refresh
- (b) interrupt controllers
- (c) clock generation
- (d) power supplies

Multiplexers

31. A 16:1 mux needs how many select lines?

- (a) 2
- (b) 4
- (c) 8
- (d) 16

32. The 2:1 mux equation is:

- (a) $SA + \bar{S}B$
- (b) $\bar{S}A + SB$
- (c) $A \oplus B$
- (d) $S(A + B)$

33. In a 2:1 mux with $S = 0$ (slide convention), the output is:

- (a) A
- (b) B
- (c) 0
- (d) S

34. A 4:1 mux internally contains the equivalent of a:

- (a) 2:4 decoder
- (b) priority encoder
- (c) comparator
- (d) full adder

35. How many 2:1 muxes make an 8:1 mux?

- (a) 3
- (b) 4
- (c) 7
- (d) 8

36. A 4:1 mux with selects A, B and $D = (0, 1, 1, 0)$ computes:

- (a) AND
- (b) OR
- (c) XOR
- (d) XNOR

Adders and PLAs (beyond)

37. A full adder's carry-out is the:

- (a) parity function
- (b) majority function
- (c) XOR
- (d) NAND

38. A full adder's sum is:

- (a) $a \oplus b \oplus c_{in}$
- (b) $ab + c_{in}$
- (c) $a + b + c_{in}$
- (d) abc_{in}

39. Ripple-carry adder delay grows:

- (a) constantly
- (b) linearly with the number of bits
- (c) logarithmically
- (d) exponentially

40. A PLA consists of:

- (a) a programmable AND plane and OR plane
- (b) only NAND gates
- (c) a decoder and a mux
- (d) flip-flops

9.2 Answers and Explanations

1 b	2 c	3 b	4 c	5 a	6 b	7 b	8 b	9 c	10 b
11 b	12 c	13 b	14 b	15 c	16 c	17 c	18 b	19 b	20 b
21 b	22 b	23 a	24 c	25 b	26 b	27 b	28 c	29 a	30 b
31 b	32 b	33 a	34 a	35 c	36 c	37 b	38 a	39 b	40 a

1. **(b)** Medium Scale Integration (slide).
2. **(c)** $1 \oplus 1 = 0$ but $1 + 1 = 1$.
3. **(b)** XOR with 1 inverts.
4. **(c)** XNOR is 1 when bits are equal.
5. **(a)** Handwritten half-adder note.
6. **(b)** Any differing bit makes the OR 1.
7. **(b)** All bits equal: AND of XNORs.
8. **(b)** $1011 \oplus 1001 = 0010$.
9. **(c)** 8 inputs: $2^8 = 256$.
10. **(b)** Only $a = 1, b = 0$.
11. **(b)** Higher equal bits pass the decision down to the first difference.
12. **(c)** $n:2^n$.
13. **(b)** One-hot.
14. **(b)** Each output is a minterm.
15. **(c)** $10_2 = 2$.
16. **(c)** 2^{10} .
17. **(c)** $10_2 = 2 = \text{MUL}$.
18. **(b)** $Y = 0$ only at m_0 .
19. **(b)** OR the minterm outputs.
20. **(b)** One 2:4 + four 3:8.
21. **(b)** Inverse of a decoder.
22. **(b)** 2s bit set for indices 2, 3.
23. **(a)** Index 0 has no bit set.
24. **(c)** $Y_1 = I_2 = 1, Y_0 = I_1 = 1$: wrong code 3.
25. **(b)** Slide equation.
26. **(b)** 1 only when all inputs are 0.
27. **(b)** Highest active is I_2 : output 2 = 10.
28. **(c)** 3 X's: $2^3 = 8$.
29. **(a)** Lower inputs don't matter once a higher one is active.
30. **(b)** Choosing the most urgent request.
31. **(b)** $\log_2 16 = 4$.
32. **(b)** $S = 0$ passes A .
33. **(a)** Handwritten " $S = 0 \Rightarrow A$ ".
34. **(a)** Select minterms = decoder outputs.

- 35. **(c)** $N - 1 = 7$.
- 36. **(c)** Output 1 when $A \neq B$.
- 37. **(b)** 1 when at least two inputs are 1.
- 38. **(a)** Odd parity.
- 39. **(b)** Carry ripples through every stage.
- 40. **(a)** Programmable AND–OR.

Appendix A

Formula Sheet, Glossary and Revision

A.1 Formula sheet

XOR / XNOR	$x \oplus y = x\bar{y} + \bar{x}y$; $\overline{x \oplus y} = xy + \bar{x}\bar{y}$; $x \oplus 0 = x$, $x \oplus 1 = \bar{x}$
Half adder	$s = a \oplus b$, $c = ab$
N -bit equality	$A \neq B = \sum (A_i \oplus B_i)$; $A = B = \prod \overline{A_i \oplus B_i}$
Magnitude	$G = \sum_k (\prod_{i>k} e_i) A_k \bar{B}_k$; $L: \bar{A}_k B_k$; $E = \prod e_i$; $e_i = \overline{A_i \oplus B_i}$
Decoder $n:2^n$	$Y_i = m_i$; one-hot; 2^n AND $n + n$ NOT
Function via decoder	$F = \sum m(S) = \sum_{i \in S} Y_i$ (or $\sum_{i \in S'} \bar{Y}_i$)
Encoder $2^N:N$	$Y_j = \sum_{k: \text{bit } j \text{ of } k=1} I_k$
4:2 priority encoder	$Y_1 = I_3 + I_2$; $Y_0 = I_3 + \bar{I}_2 I_1$; $\text{None} = \bar{I}_3 + \bar{I}_2 + \bar{I}_1 + \bar{I}_0$
Mux $N:1$	$\log_2 N$ selects; $Y = \sum_i m_i(S) D_i$; 2:1: $Y = \bar{S}A + SB$
Mux tree	$N - 1$ two-to-one muxes, $\log_2 N$ levels
Full adder	$s = a \oplus b \oplus c_{in}$; $c_{out} = ab + c_{in}(a \oplus b)$

A.2 Glossary

Comparator Block outputting $A = B$ (and possibly $A > B$, $A < B$).

Decoder n inputs, 2^n one-hot outputs (one per minterm).

Demultiplexer Routes one input to one of N outputs.

Enable Input that turns a block's outputs on/off.

Encoder 2^N one-hot inputs to an N -bit code.

LUT Lookup table: a mux fed by stored bits (FPGAs).

MSI Medium-scale integration: tens to hundreds of transistors.

Multiplexer Selects one of N inputs using $\log_2 N$ select bits.

One-hot Exactly one signal in a group is 1.

PLA Programmable AND plane + OR plane.

Priority encoder Encoder that reports the highest-index active input, plus a None/valid output.

Ripple-carry adder Chain of full adders.

A.3 Revision checklist

- ☐ Why blocks (MSI, abstraction); list the standard blocks.
- ☐ XOR/XNOR tables and equations; XOR in the half adder.
- ☐ 1-bit and N -bit equality comparator; example 11 vs 9.
- ☐ Derive $A > B$ from the MSB down.

- ☐ 2:4 decoder circuit and table; one-hot; address and opcode decoding.
- ☐ Implement a function with a decoder; the $A + B + C + D$ exercise.
- ☐ 4:2 encoder equations and their one-hot assumption.
- ☐ Priority encoder table, input don't-cares, equations, None.
- ☐ 2:1 and 4:1 mux equations; $\log_2 N$; mux trees; mux as lookup table.
- ☐ (Beyond) full adder, ripple carry, PLA.